4️⃣

Round 4 - “The More The Merrier”

For this second round of the Great Orbital Ascension Trials, the **Frontier Trade Watch** (FTW) has disclosed information about the counterparties active in the market. Their IDs have been added to the historical trade data available in the Data Capsule.

You will continue trading ***Hydrogel Packs*** (`HYDROGEL_PACK`), ***Velvetfruit Extract*** (`VELVETFRUIT_EXTRACT`), and ***10 Velvetfruit Extract Vouchers*** (`VELVETFRUIT_EXTRACT_VOUCHER`). This time, however, having insight into your counterparties, and understanding what defines their trading behavior and the unique opportunities they bring, could shift the balance for teams that know how to separate profit from pretense.

In addition to your algorithmic trading activities, you will also have the opportunity to manually trade the ***Aether Crystal***, along with ***a collection of option contracts*** based on it. Some of these contracts are more exotic than others. You must determine a strategy that turns this one-time opportunity into profit.

Be aware that these exotic options operate independently from your algorithmic trading activities.

Round Objective

Optimize your Python program to trade `HYDROGEL_PACK`, `VELVETFRUIT_EXTRACT`, and `VELVETFRUIT_EXTRACT_VOUCHER`, incorporating the newly disclosed counterparty information into your strategy.

Select from the available Aether Crystal and corresponding option contracts, and submit your orders to generate additional profit.

Algorithmic trading challenge: “Hello, I’m Mark”

The products are the same as in Round 3 (`HYDROGEL_PACK`, `VELVETFRUIT_EXTRACT`, and 10 `VELVETFRUIT_EXTRACT_VOUCHER` options), but now you have counterparty information available. That is, you can identify every other participant in the market and study their behavior.

In the datamodel described in Appendix B: datamodel.py file in 💻 **Writing an Algorithm in Python**, you will find the `Trade` class defined. For previous rounds 1,2,3, the `self.buyer` and `self.seller` fields were always `None` as no counterparty information was available.

Code snippet for `class Trade`:

```
class Trade:
    def __init__(self, symbol: Symbol, price: int, quantit
y: int, buyer: UserId = None, seller: UserId = None, timest
amp: int = 0) -> None:
        self.symbol = symbol
        self.price: int = price
        self.quantity: int = quantity
        self.buyer = buyer
        self.seller = seller
        self.timestamp = timestamp

    # Some methods
```

With increased transparency in the market, however, these `self.buyer` and `self.seller` fields now represent the names of the participants! Please feel free to leverage this information however you see fit, and refine your strategy using this extra visibility!

The position limits (see the Position Limits page for extra context and troubleshooting) are still

- `HYDROGEL_PACK`: 200
- `VELVETFRUIT_EXTRACT`: 200
- `VELVETFRUIT_EXTRACT_VOUCHER`: 300 for each of the 10 vouchers.

Example (building on the example from Round 3): `VEV_5000` is an option with strike price 5000, has TTE=4 days in round 4, and has a position limit of 300.

Manual trading challenge: “Vanilla Just Isn’t Exotic Enough”

As the Intarian economy evolved, trading expanded beyond standard calls and puts. In this round, you can trade `AETHER_CRYSTAL`, vanilla options with 2 and 3 week expiries, and several exotic derivatives written on the same underlying.

Please note that a ‘week’ here refers to 5 trading days and that the ‘standard’ number of trading days per year is 252 (since some big exchanges are typically open 252 days per year). So “2 weeks” means 10 trading days, and “3 weeks” represents 15 trading days. For transparency purposes, this is how the days are computed on our end:

```
TRADING_DAYS_PER_YEAR = 252
STEPS_PER_DAY = 4
STEPS_PER_YEAR = TRADING_DAYS_PER_YEAR * STEPS_PER_DAY

def weeks_to_years(weeks: float) -> float:
    # 5 business days per week, annualized to 252 trading d
ays
    return (weeks * 5) / TRADING_DAYS_PER_YEAR

def steps_for_weeks(weeks: float) -> int:
    return int(round(weeks * 5 * STEPS_PER_DAY))
```

Thus, when you see "2 weeks", assume it means `2 * 5 * STEPS_PER_DAY` steps over 10 days.

Your objective is to construct positions that generate positive expected PnL. But be aware: unhedged exposure can lead to large losses, so risk management matters. The PnL is marked to the ‘fair’ value upon expiry, which

is the average value of the product across 100 simulations. You should maximize the PnL on the products *as you hold them till expiry if you buy* (and short till expiry if you short). In other words, there is no buying or selling across days. You decide to buy/sell at t=0 (start of round 4) and hold it till expiry, at which point they are marked against their fair value. ***This means this challenge is completely standalone (there is NO relationship to Round 1)***.

All products are written on `AETHER_CRYSTAL`. You can trade the underlying, 2 week and 3 week vanilla calls and puts, and the following exotics:

❓ Chooser Option

Expires in 3 weeks. After 2 weeks, the buyer chooses whether it becomes a call or a put, selecting whichever would be in the money at that time. It then behaves like a standard option for the final week until expiry.

🔀 Binary Put Option

Has an all-or-nothing payoff. If the underlying is below the strike at expiry, it pays the specified amount. Otherwise, it expires worthless.

🥊 Knock-Out Put Option

Behaves like a regular put unless the underlying ever trades below the knockout barrier before expiry. If the barrier is breached at any point, the option immediately becomes worthless.

You may buy or sell up to the displayed volume in each product. Note that the “contract size” is 3000 across all products, and is only used as a way to scale PnL proportionally to Rounds 3 and 5; think of it as a PnL multiplier on the PnL you make on the individual products (underlying, options) listed in the table. The prices you see are for each individual option.

Your final score is the average PnL across 100 simulations of the underlying.

The underlying `AETHER_CRYSTAL` is simulated using Geometric Brownian Motion with zero risk-neutral drift and fixed annualized volatility of 251%. Prices evolve on a discrete grid of 4 steps per trading day, assuming 252 trading days per

year (see code above). There is no ‘continuous’ modeling under the hood that could trigger a knock-out; you should only consider these discrete points.

And remember, when payoffs become conditional, so does risk. Good luck!

Note on “price” column: this is purely cosmetic and should show the ‘investment cost’, but is unrelated to your PnL. It should in no way afffect your trading decision, and you can freely ignore it.

Submit your orders

Input your orders for the Aether Crystal and corresponding option contracts directly in the Manual Challenge Overview window and click the “Submit” button. You can re-submit new orders until the end of the trading round. When the round ends, the last submitted orders will be locked in and processed.